

Toka Priyanka

tokapriyanka@gmail.com

AI/ML intern

Role: Computer Vision

AI Model Optimization Report

1. Introduction

AI model optimization is an essential process for improving the performance, speed, scalability, and accuracy of the virtual try-on system. The optimization stage focuses on enhancing pose estimation, segmentation quality, inference efficiency, recommendation systems, and deployment readiness.

This phase investigates landmark detection methods, lightweight models, inference acceleration techniques, and segmentation optimization approaches to improve overall system performance.

The uploaded optimization notes emphasize improving speed, accuracy, and deployment scalability of AI pipelines.

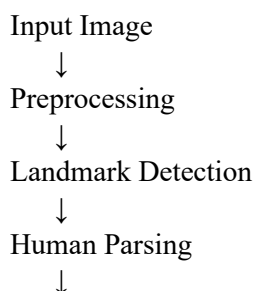
2. Objectives

The objectives of AI optimization are:

- Improve landmark detection accuracy
- Reduce try-on processing time
- Increase model inference speed
- Optimize segmentation quality
- Improve recommendation systems
- Reduce latency
- Improve deployment efficiency

3. Current AI Pipeline

Current optimization workflow:



Garment Alignment



Rendering



Recommendation Module



Final Output

Optimization targets:

- Accuracy
- Speed
- Memory usage
- GPU efficiency

4. Landmark Detection Optimization

Objective

Improve body landmark accuracy for better garment fitting.

Landmark detection directly affects:

- Sleeve alignment
- Torso fitting
- Garment deformation
- Pose robustness

OpenPose Optimization

OpenPose detects multiple body keypoints.

Detected landmarks:

- Shoulder
- Elbow
- Wrist
- Hip
- Knee
- Neck

Advantages:

- High precision
- Multi-point detection

- Better fitting quality

Applications:

- Garment alignment
- Pose estimation
- Body mapping

The uploaded notes mention OpenPose as an important pose estimation technique.

MediaPipe Optimization

MediaPipe provides lightweight body tracking.

Benefits:

- Fast inference
- Low latency
- Real-time execution
- Lightweight architecture

Advantages:

- Mobile compatibility
- Reduced computation
- Faster response time

MediaPipe optimization is highlighted for real-time systems.

Landmark Comparison

Model	Accuracy	Speed	Usage
-------	----------	-------	-------

OpenPose	High	Medium	Precise fitting
----------	------	--------	-----------------

MediaPipe	Medium	High	Real-time tracking
-----------	--------	------	--------------------

5. Inference Speed Optimization

Objective

Reduce execution time and improve responsiveness.

Performance metrics:

- Latency
- FPS
- Memory usage

- Throughput

ONNX Optimization

ONNX converts models into optimized execution formats.

Benefits:

- Faster execution
- Cross-platform support
- Reduced overhead
- Better deployment

TensorRT Acceleration

TensorRT improves GPU inference.

Capabilities:

- Reduced latency
- GPU optimization
- Faster prediction
- Lower memory consumption

The uploaded optimization notes mention ONNX and TensorRT for latency reduction.

6. Lightweight Model Optimization

Objective

Enable deployment on lightweight devices.

Requirements:

- Reduced computation
- Mobile compatibility
- Faster processing

MobileNe

MobileNet uses depthwise convolutions.

Advantages:

- Lightweight architecture
- Faster inference
- Mobile support

Applications:

- Edge devices
- Mobile deployment

EfficientNet

EfficientNet improves performance while reducing computational cost.

Benefits:

- Better accuracy
- Lower parameters
- Improved efficiency

The uploaded notes recommend EfficientNet and MobileNet for lightweight deployment.

Model Comparison

Model	Parameters	Speed	Deployment
OpenPose	High	Medium	GPU
MediaPipe	Low	High	Real-time
MobileNet	Low	High	Mobile
EfficientNet	Medium	Medium	Balanced

7. Human Parsing Optimization

Objective

Improve body segmentation accuracy.

Problems observed:

- Arm overlap
- Hair interference
- Background leakage

Detectron2 Optimization

Detectron2 improves segmentation precision.

Capabilities:

- Instance segmentation

- Better body separation
- Improved masks
- Occlusion handling

Benefits:

- Improved parsing quality
- Better garment layering

The uploaded notes state Detectron2 improves human parsing quality.

8. Recommendation System Optimization

Objective

Improve personalized outfit suggestions.

Input factors:

- Clothing category
- Body type
- User preference
- Size information

Recommendation Flow

User Input



Body Analysis



Clothing Features



Recommendation Engine



Suggested Outfit

Outputs:

- Outfit suggestions
- Size recommendation
- Similar products
- Personalized styling

The uploaded notes highlight recommendation systems for personalization.

9. Deployment Architecture

Proposed optimized architecture:

Frontend



Flask API



Optimized AI Models



GPU Inference Engine



Rendering Module



Recommendation Engine

10. Challenges

Major optimization challenges:

Hardware Issues

- High GPU usage
- Memory bottlenecks
- Processing load

Model Issues

- Slow inference
- Large model size
- Deployment difficulty

Segmentation Issues

- Occlusion
- Background noise
- Body overlap

The uploaded notes identify GPU usage and memory bottlenecks as major issues.

11. Future Improvements

Planned enhancements:

- Quantization
- Pruning techniques
- Edge deployment
- Better recommendation logic

- Real-time optimization
- Lightweight segmentation models
- Multi-GPU execution

12. Conclusion

The AI optimization process improved the efficiency and scalability of the virtual try-on system by enhancing landmark detection, segmentation quality, inference performance, and recommendation mechanisms.

The optimization techniques reduced latency, improved deployment readiness, and established a scalable architecture suitable for real-time virtual try-on applications.